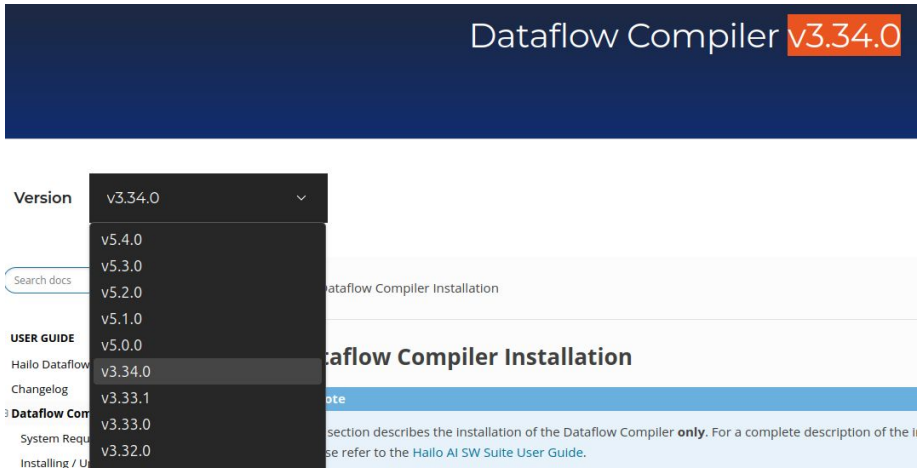
AI Accelerator Software Suite Setup for Hailo NPU

Lab AI Programming, Software Engineering, KMITL 2026

Hailo Dataflow Compiler (DFC) Version

- For the current hardware (Hailo-8) in this lab, **use v3.34.0**
- The v5.4.0 or higher are for Hailo-10/Hailo-10L which are not available in lab.
- Document link: https://hailo.ai/developer-zone/documentation/dataflow-compiler-v3-34-0/?sp_referrer=install/install.html



Outline

1. Introduction to AI Accelerator Hardware
2. Workflow for deploy AI locally on embedded device.
3. Introduction to Hailo AI Accelerator
    a. Hailo NPU (Hailo-8/8L, Hailo-10)
    b. Hailo Ecosystem
        i. Hailo Dataflow Compiler
4. Lab 8: Use AI accelerator HW on MNIST model
    a. Model Compilation & Inference
    b. Performance Comparison / FPS
    c. Real-time Inferencing on RPi camera module

What is the AI Hardware Accelerator

- Hardware specially designed for handling **fast** and **energy-efficient** AI computation, covering different types of AI algorithms and hardware platforms
- List of Accelerator:
  - Tensor Processing Unit (TPU)
  - Neural Processing Unit (NPU)
- Reading reference: https://arxiv.org/pdf/2411.13717

Inside AI Hardware Accelerator

Example NPU

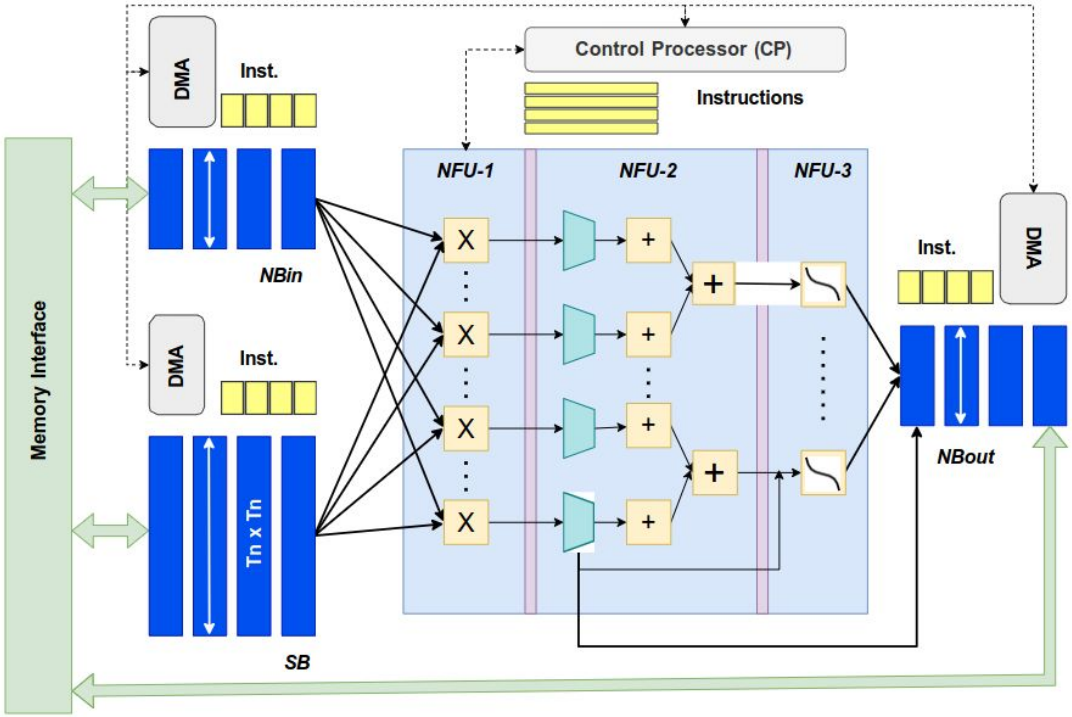


Fig. 1.5 DaDianNao Accelerator Architecture (the figure is recreated from [20]).

Inside AI Hardware Accelerator

Example NPU

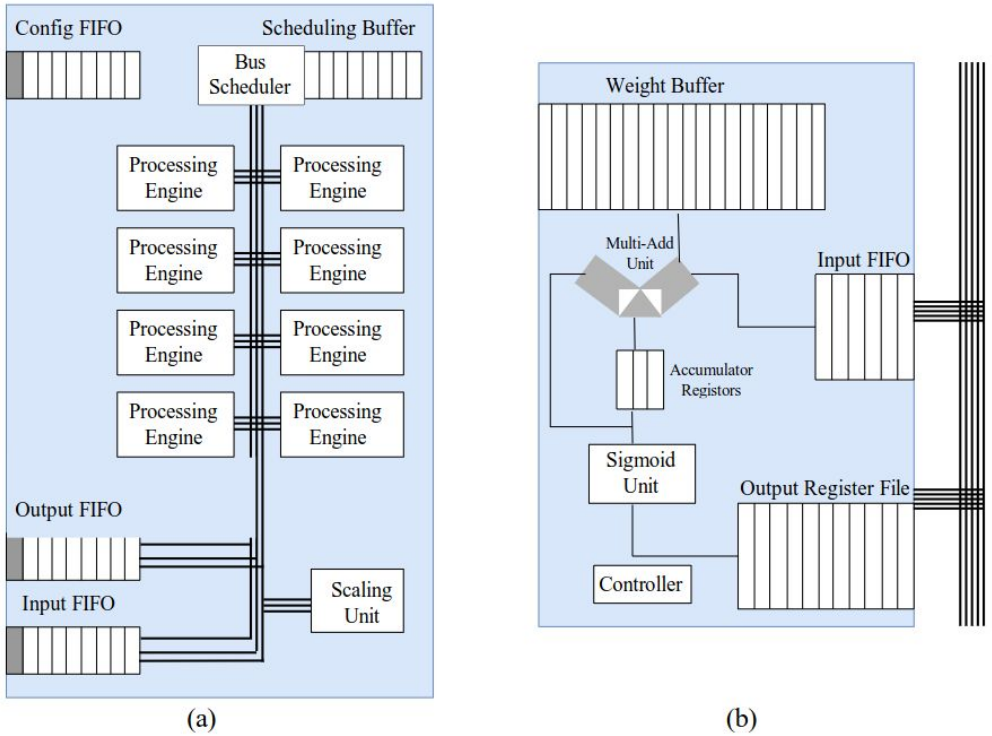
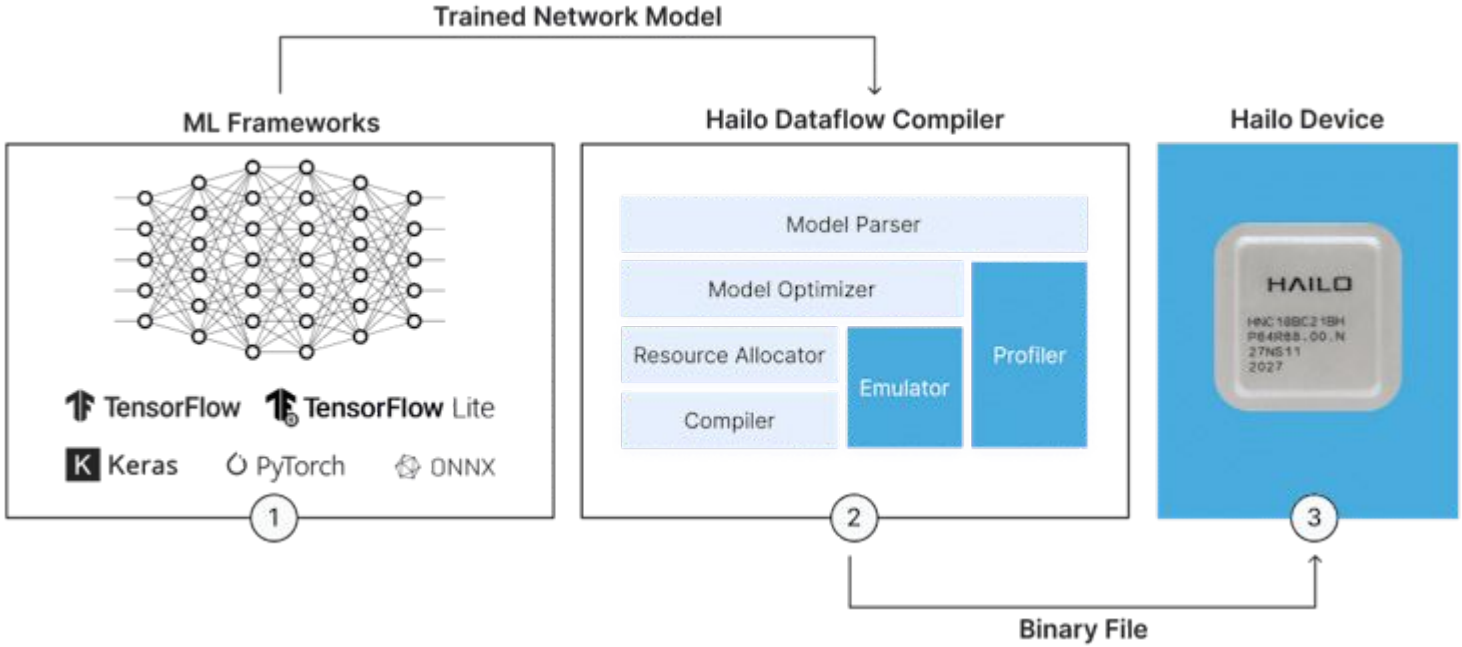
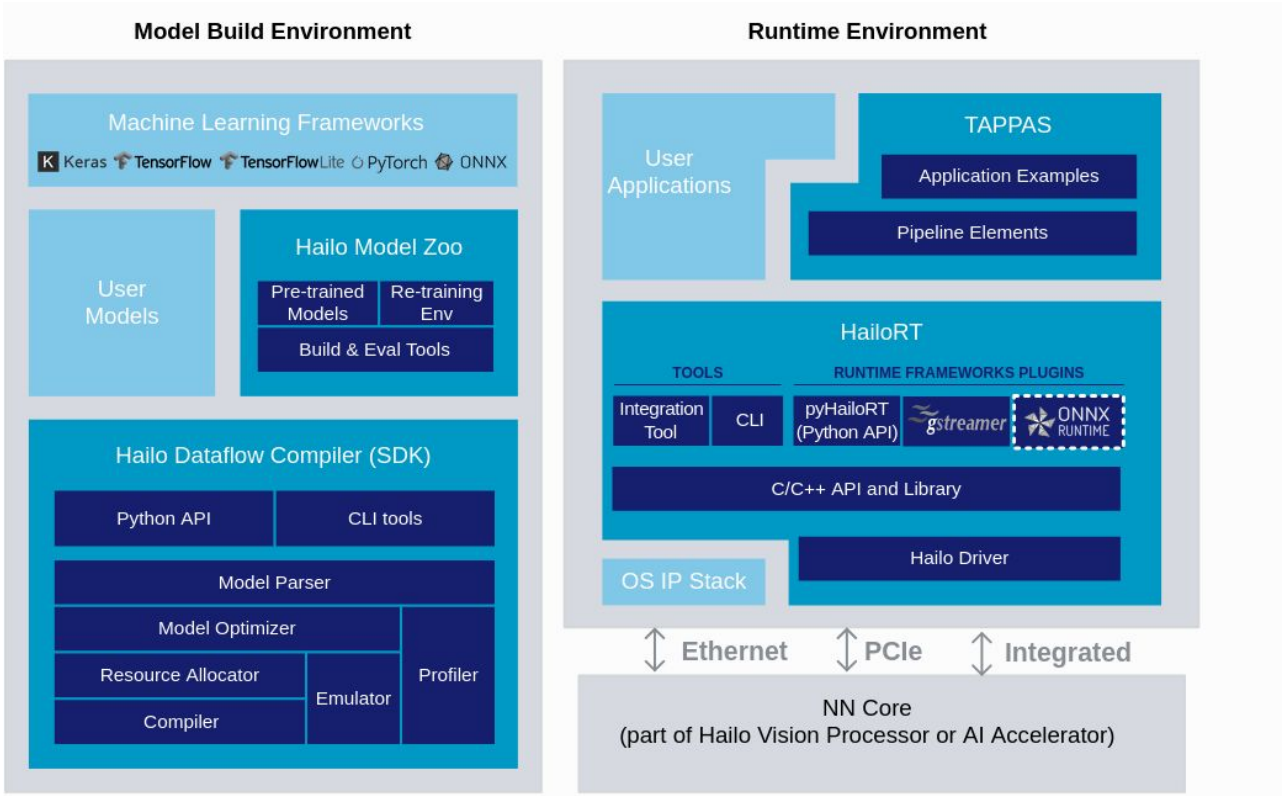


Fig. 1.6 The Neural Processing Unit (NPU) Architecture (a) 8-Processing engine (PE) NPU, (b) Single processing engine (PE) (the figure is recreated from [17]).

Hailo NPU Development Workflow



Hailo NPU Development Workflow



Hailo NPU Development Workflow

Product	Hailo-10	Hailo-8		Hailo-8L
Memory	8 GB	32 MB		32 MB
Peak TOPS	40 TOPS	26 TOPS		13 TOPS
PCIe interface	PCIe Gen 3 (4 lanes)	PCIe Gen 3 (4 lanes)	PCIe Gen 3 (2 lanes)	PCIe Gen 3 (2 lanes)
Modules	M.2 (M-key)	M.2 (M key)	M.2 (B+M, A+E)	M.2 (B+M, A+E)



Hailo Parser

- Create the Hailo's metadata file (.har) from .pt/.tf/.onnx file
- Hailo CMD
  - `$ hailo parser onnx /path/to/model.onnx --hw-arch hailo8`

- Hailo Python API
  - See https://hailo.ai/developer-zone/documentation/dataflow-compiler-v3-34-0/?sp_referrer=tutorials_notebooks/notebooks/DFC_1_Parsing_Tutorial.html
  - Or **onnx2har.py** for an example python script in the lab sheet.

```
runner = ClientRunner(hw_arch=chosen_hw_arch)
hn, npz = runner.translate_onnx_model(
    onnx_path,
    onnx_model_name,
    start_node_names=["input.1"],
    end_node_names=["192"],
    net_input_shapes={"input.1": [1, 3, 224, 224]},
)
```

Hailo Parser

- Hailo Python API
  - onnx2har.py
  - config.json

```
{
        "hailo_arch":"hailo8",
        "onnx_path":"../models/model_mnist.onnx",
        "model_name":"model_mnist",
        "name_start":"input",
        "name_end":"output",
        "size_input":"1,784",
        "output_path":"output/",
        "har_name":"model_mnist.har",
        "quantized_har_name":"model_mnist.har",
    "calib_set":"../calib_set.npy"
}
```

```python
from IPython.display import SVG
from hailo_sdk_client import ClientRunner
import argparse
import json
import os



parser = argparse.ArgumentParser()
# Adding optional argument
parser.add_argument("-c", "--cfg", help="ONNX 2 HAR configuration file as .json file", default='.json')
args = parser.parse_args()
f = open(args.cfg)
data = json.load(f)
chosen_hw_arch = data['hailo_arch']
onnx_path = data['onnx_path']
print(chosen_hw_arch)
print(onnx_path)
f.close()

if not os.path.exists(data['output_path']):
    os.mkdir(data['output_path'])

# Read input size of model
size_input = data['size_input']
size_list = size_input.split(",")
size_input = [int(x) for x in size_list]
print("input size=", size_input)

# Run Hailo Translator
runner = ClientRunner(hw_arch=chosen_hw_arch)
hn, npz = runner.translate_onnx_model(
    onnx_path,
    data["model_name"],
    start_node_names=data["name_start"],
    end_node_names=data["name_end"],
    net_input_shapes={data["name_start"]: size_input},
)

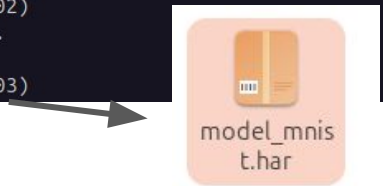
runner.save_har(data["output_path"] + data["model_name"] + ".har")
```

Hailo Parser

Example Output

- `$ hailo parser onnx /path/to/model.onnx --hw-arch hailo8`

```
[info] No GPU chosen and no suitable GPU found, falling back to CPU.
[warning] Cannot use graphviz, so no visualizations will be created
[info] Current Time: 16:06:46, 10/24/26
[info] CPU: Architecture: x86_64, Model: AMD Ryzen 7 260 w/ Radeon 780M Graphics, Number Of Cores: 16, Utilization: 1.0%
[info] Memory: Total: 14GB, Available: 10GB
[info] System info: OS: Linux, Kernel: 7.0.0-38-generic
[info] Hailo DFC Version: 3.34.0
[info] HailoRT Version: Not Installed
[info] PCIe: No Hailo PCIe device was found
[info] Running `hailo parser onnx ../models/model_mnist.onnx --hw-arch hailo8`
[info] Translation started on ONNX model model_mnist
[info] Restored ONNX model model_mnist (completion time: 00:00:00.01)
[info] Extracted ONNXRuntime meta-data for Hailo model (completion time: 00:00:00.02)
[info] Start nodes mapped from original model: 'input': 'model_mnist/input_layer1'.
[info] End nodes mapped from original model: '/layer_out/Gemm'.
[info] Translation completed on ONNX model model_mnist (completion time: 00:00:00.03)
```



Hailo Optimizer

- To prepare a trained NN model for real-time, low-power execution on Hailo hardware by performing **quantization** and hardware-specific **optimization**
  - Converting **float32** datatype input layer into **int8** for Hailo HW
  - Normalization for Hailo HW
  - etc.
- Hailo CMD
  - `$ hailo optimize /path/to/model.har --hw-arch hailo8 --calib-dataset /path/to/calibration_data.npy`
  - If no calibration dataset (Do not recommended)
    `$ hailo optimize /path/to/model.har --hw-arch hailo8 --use-random-calib-set`

- Hailo Python API
  - See Or **optimize.py** for an example python script in the lab sheet.

Hailo Optimizer

- Creating calibration file (.npy)
  - Use training dataset of the model, pack them into the tensor variable with same dimension of training data. Then export the tensor variable into .npy
  - See **create_model_calib.py**

```
batch_size = 1
# Read the MNIST dataset by using torch and torchvision
# This will download the file to PATH if there's no dataset in the PATH
train_loader = DataLoader(
    MNIST('dataset', train=True, download=True,
          transform=torchvision.transforms.Compose([torchvision.transforms.ToTensor()])),
    batch_size=batch_size,
    shuffle=True
)
# Start creating calibration file from MNIST dataset as the input for our model
# 28x28 images with 60,000 image set
print("Start creating calibration file")
import numpy as np
X = np.empty((60000, 784))
for i in range(60000):
    train_features, train_labels = next(iter(train_loader))
    train_features = flatten_layer(train_features)
    x = train_features.numpy()
    X[i, :] = x
print(X)
print("Finished")
np.save("calib_set.npy", X)
```

Hailo Optimizer

Example Output

```
[warning] Reducing optimization level to 0 (the accuracy won't be optimized and compression won't be used) because there's no available GPU
[warning] Running model optimization with zero level of optimization is not recommended for production use and might lead to suboptimal accuracy results
[info] Model received quantization params from the hn
[info] MatmulDecompose skipped
[info] Starting Mixed Precision
[info] Model Optimization Algorithm Mixed Precision is done (completion time is 00:00:00.02)
[info] LayerNorm Decomposition skipped
[info] Starting Statistics Collector
[info] Using dataset with 64 entries for calibration
Calibration: 100%|                                                                                | 64/64 |
[info] Model Optimization Algorithm Statistics Collector is done (completion time is 00:00:00.48)
[info] Starting Fix zp_comp Encoding
[info] Model Optimization Algorithm Fix zp_comp Encoding is done (completion time is 00:00:00.00)
[info] Matmul Equalization skipped
[info] Starting MatmulDecomposeFix
[info] Model Optimization Algorithm MatmulDecomposeFix is done (completion time is 00:00:00.00)
[info] Finetune encoding skipped
[info] Bias Correction skipped
[info] Adaround skipped
[info] Quantization-Aware Fine-Tuning skipped
[info] Layer Noise Analysis skipped
[info] Model Optimization is done
```



Hailo Compiler

- Compile the .har file after optimization into **.hef** executable file. It will be used for inference the model along with Hailo-RT
- Hailo CMD
  - `$ hailo compiler /path/to/model_quantized.har --hw-arch hailo8 --output-dir .`

- Hailo Python API
  - See Or **compile.py** for an example python script in the lab sheet.

Hailo Compiler

Example Output




```
[info] Validating layers feasibility

Validating model_mnist_context_0 layer by layer (100%)

● Finished

[info] Layers feasibility validated successfully
[info] Running resources allocation (mapping) flow, time per context: 59m 59s
Context:0/0 Iteration 4: Trying parallel mapping...
          cluster_0  cluster_1  cluster_2  cluster_3  cluster_4  cluster_5  cluster_6  cluster_7  prepost
 worker0  *          *          *          *          V          V          *          *          V
 worker1
 worker2
 worker3  V          *          *          *          V          V          V          *          V

  00:00
Reverts on cluster mapping: 0
Reverts on inter-cluster connectivity: 0
Reverts on pre-mapping validation: 0
Reverts on split failed: 0

[info] Iterations: 4
Reverts on cluster mapping: 0
Reverts on inter-cluster connectivity: 0
Reverts on pre-mapping validation: 0
Reverts on split failed: 0
[info] +-----------+--------------------+--------------------+--------------------+
[info] | Cluster   | Control Utilization | Compute Utilization | Memory Utilization |
[info] +-----------+--------------------+--------------------+--------------------+
[info] | cluster_4 | 93.8%              | 23.4%              | 35.2%              |
[info] | cluster_5 | 62.5%              | 15.6%              | 21.9%              |
[info] +-----------+--------------------+--------------------+--------------------+
[info] | Total     | 19.5%              | 4.9%               | 7.1%               |
[info] +-----------+--------------------+--------------------+--------------------+
[info] Successful Mapping (allocation time: 1s)
[info] Compiling kernels of model_mnist_context_0...
[info] Bandwidth of model inputs: 0.00598145 Mbps, outputs: 7.62939e-05 Mbps (for a single frame)
[info] Bandwidth of DDR buffers: 0.0 Mbps (for a single frame)
[info] Bandwidth of inter context tensors: 0.0 Mbps (for a single frame)
[info] Building HEF...
[info] Successful Compilation (compilation time: 0s)
```